AI Research & Knowledge Assistant

---

1. Background

Modern organizations deal with thousands of research papers, technical documents, internal documentation, product specifications, and knowledge bases. Finding accurate information from these documents is often time-consuming and inefficient.

Traditional keyword search systems frequently fail to understand context, while generic Large Language Models (LLMs) may generate inaccurate or hallucinated responses when asked about domain-specific information.

Organizations are increasingly adopting **Retrieval-Augmented Generation (RAG)** systems to enable AI assistants that can retrieve relevant information from enterprise knowledge bases before generating responses.

Your task is to design and implement an **AI Research & Knowledge Assistant** capable of:

- Understanding multiple documents
- Retrieving relevant information
- Reasoning over retrieved content
- Generating grounded responses with proper citations

In addition to document intelligence, the system should leverage Machine Learning to classify documents and provide analytical insights.

---

2. Objective

Develop a **production-oriented backend application** that enables users to:

- Upload research papers or technical documents
- Search across multiple documents using semantic retrieval
- Interact with an AI assistant
- Compare documents
- Summarize information
- Classify uploaded documents using a TensorFlow model

The application should demonstrate strong software engineering practices while integrating modern AI technologies such as embeddings, vector databases, Large Language Models, and Retrieval-Augmented Generation.

---

3. Problem Statement

You have been hired to build an AI-powered research assistant for an organization that maintains a large collection of technical documents.

The organization requires a system capable of:

#	Capability
1	Understanding multiple uploaded documents
2	Retrieving relevant information using semantic search
3	Generating accurate responses supported by citations
4	Comparing information across multiple documents
5	Summarizing lengthy documents
6	Classifying documents into predefined categories
7	Maintaining conversation context during user interactions

The application should expose **REST APIs** and be designed using clean, modular, and scalable architecture.

---

4. Functional Requirements

4.1 Document Management

The application should provide functionality to manage documents.

Required Features

- Upload one or more PDF documents
- Store document metadata
- List all uploaded documents

- Delete uploaded documents
- Reprocess documents when required

Document Metadata

Each uploaded document should maintain information such as:

Field	Description
Document ID	Unique identifier for the document
Document Name	Original file / display name
Upload Timestamp	Date and time of upload
Total Pages	Page count of the source PDF
Total Chunks	Number of chunks produced by the pipeline
Processing Status	Current state of the processing pipeline

4.2 Document Processing Pipeline

Each uploaded document should automatically undergo preprocessing.

The processing pipeline should include:

1. **Text extraction**
2. **Data cleaning**
3. **Intelligent text chunking**
4. **Embedding generation**
5. **Vector database indexing**

> Students should choose an appropriate chunking strategy and **justify their implementation** in the documentation.

4.3 Semantic Search

The application should support semantic retrieval instead of simple keyword matching.

Given a user query, the system should:

1. Generate embeddings for the query
2. Retrieve the most relevant document chunks

3. Rank retrieved results
4. Return the most relevant information

The implementation should support searching **across multiple uploaded documents**.

4.4 AI Question Answering

Develop an AI-powered question answering service using **Retrieval-Augmented Generation**.

The assistant should answer questions based **only** on retrieved document context.

The generated response should include:

- Final Answer
- Source Document(s)
- Page Number(s)
- Retrieved Context
- Confidence Score *(optional)*

  If sufficient information is unavailable, the assistant should clearly communicate that the answer could not be determined from the available documents.

4.5 Document Comparison

The assistant should support comparison between two or more uploaded documents.

Example use cases:

- Compare methodologies
- Compare advantages and disadvantages
- Identify similarities
- Identify differences
- Compare conclusions
- Compare implementation approaches

The response should be generated using information retrieved from **all selected documents**.

4.6 Document Summarization

Provide document summarization capabilities.

The application should generate:

- Executive Summary
- Technical Summary
- Bullet Point Summary
- Key Takeaways

The summaries should be generated using an LLM.

4.7 TensorFlow Document Classification

Train a TensorFlow model capable of classifying uploaded documents into predefined categories.

Students may choose appropriate public datasets or prepare their own labelled dataset.

Example categories:

- Artificial Intelligence
- Machine Learning
- Computer Vision
- Natural Language Processing
- Robotics
- Cyber Security
- Cloud Computing

The solution should include:

Stage	Requirement
1	Data preprocessing
2	Feature engineering
3	Model training
4	Model evaluation
5	Model persistence

6 Prediction API

The trained model should **automatically classify newly uploaded documents**.

4.8 Conversation Memory

The assistant should maintain conversational context throughout a user session.

Example:

User: Summarize Paper A.
User: What are its limitations?

The assistant should correctly identify that *"its"* refers to **Paper A** without requiring the user to repeat the document name.

4.9 Search Modes

The application should support multiple search strategies, for example:

- Keyword Search
- Semantic Search
- Hybrid Search *(optional)*

Students should explain **when each search strategy is most appropriate**.

4.10 Analytics

Provide basic analytics about the uploaded knowledge base, for example:

- Number of uploaded documents
- Number of processed chunks
- Total embeddings generated
- Most queried documents
- Total questions answered

---

5. Non-Functional Requirements

The solution should demonstrate:

- Modular architecture
- Reusable components
- Proper logging
- Error handling
- Environment-based configuration
- Scalability
- Readable code
- Production-ready project structure

---

7. Expected Deliverables

Students are expected to submit:

- Complete source code
- Public GitHub repository
- README documentation
- Trained TensorFlow model
- Sample documents
- API documentation
- Screenshots demonstrating functionality
- Postman Collection or Swagger documentation

---

8. Documentation Requirements

The README should include:

- Project Overview
- Architecture Diagram
- Technology Stack
- Setup Instructions
- Environment Variables
- API Documentation
- Assumptions
- Design Decisions
- Limitations
- Future Improvements

---

10. Bonus Features (Optional)

Candidates may implement additional features such as:

Category	Features
Security & Access	Authentication & Authorization, Multi-user support
Retrieval & AI	Streaming LLM responses, Hybrid Retrieval (BM25 + Vector Search), Reranking models, Agent-based architecture
Document Handling	Multi-modal document support (images, tables), OCR for scanned PDFs
Engineering	Caching, Dockerization, CI/CD Pipeline, Cloud Deployment, Unit & Integration Testing

---

11. Submission Guidelines

- Submit the GitHub repository link before the deadline.
- Ensure the project is fully executable using the provided README.
- **Do not commit API keys or sensitive credentials.**
- Include a `.env.example` file with placeholder values.
- The repository should contain all required documentation and assets.

---

12. Important Note

You are encouraged to use publicly available libraries, frameworks, and AI-assisted development tools.

However, during the technical interview, you will be expected to:

- Explain your implementation
- Justify design decisions
- Modify portions of your code

The evaluation will prioritize your **understanding of the system, code quality, architectural decisions, and problem-solving approach** over the use of any specific library or framework.